



UCCD2203 Database Systems

Lecture 3

By

Mr. Cheang Kah Wai, Alvin

Department of Information Systems, FICT

Lecture 3 Topic: Database Design Methodology - Part 1

- Introduction to Database Design Methodology
- Conceptual Database Design
- Logical Database Design

Database Design

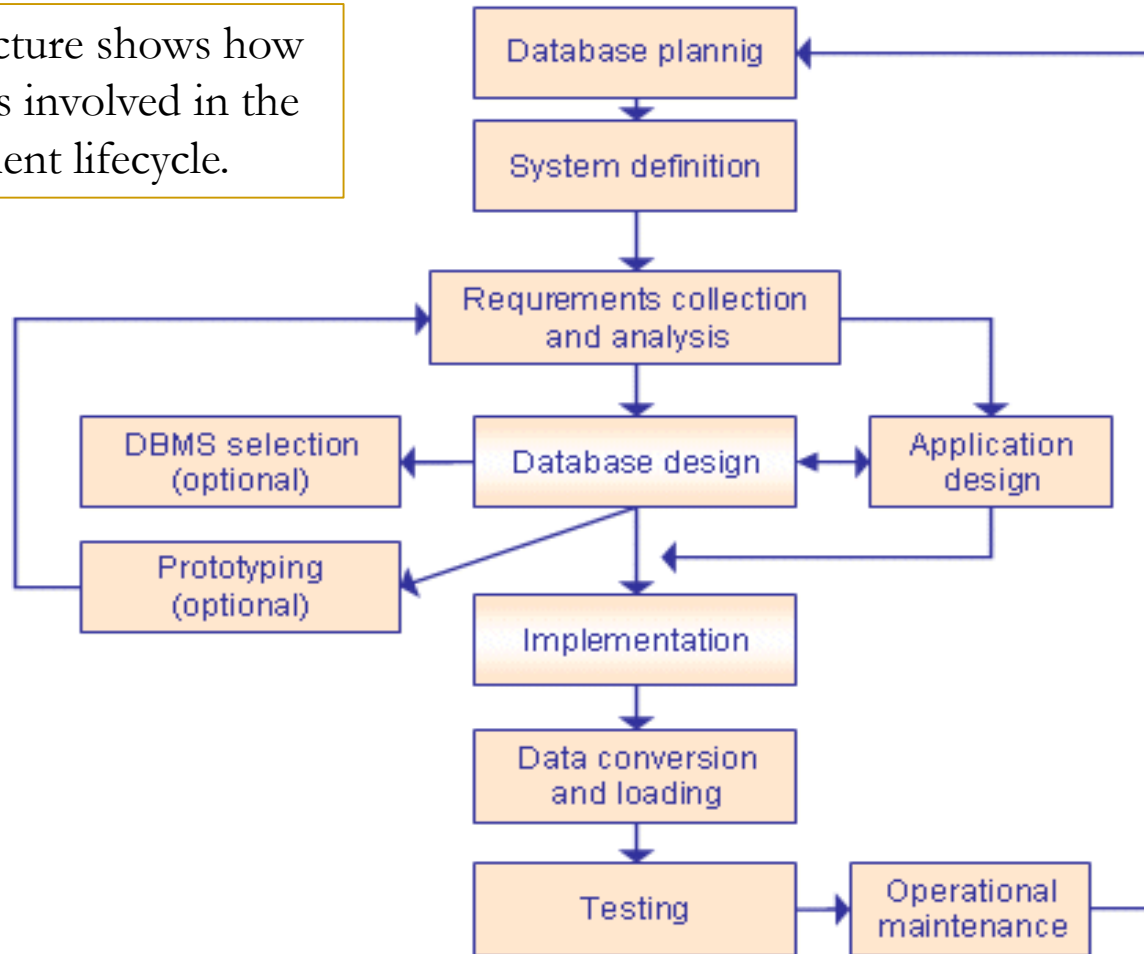
- A database is usually a fundamental component of the information system, especially in business-oriented systems. Thus, database design is part of system development.

Design Methodology

- A structured approach that uses procedures, techniques, tools and documentation aids to support and facilitate the process of design.
- A design methodology consists of phases each containing a number of steps that guide the designer in the techniques appropriate at each stage of the project.

Stages of Database System Development Life cycle

The following picture shows how database design is involved in the system development lifecycle.



Database Planning

The **database planning** includes the activities that allow the stages of the database system development lifecycle to be realized as efficiently and effectively as possible. This phase must be integrated with the overall Information System strategy of the organization.

The very first step in database planning is to define the mission statement and objectives for the database system. That is the definition of:

- the major aims of the database system,
- the purpose of the database system,
- the supported tasks of the database system,
- the resources of the database system.

Requirements Collection and Analysis

During the **requirements collection and analysis phase**, the collection and analysis of the information about the part of the enterprise to be served by the database are completed. The results may include, e.g.:

- the description of the data used or generated
- the details how the data is to be used or generated
- any additional requirements for the new database system

Database Management System Selection

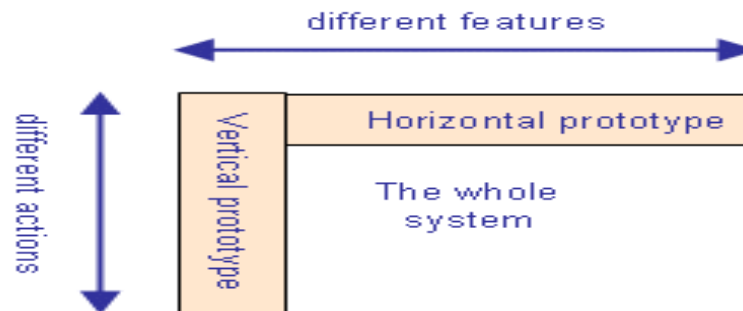
- This is an optional phase. When there is a need for a new database management system (DBMS), this phase is done. DBMS means a database system like Access, SQL Server, MySQL, Oracle.
- In this phase the criteria for the new DBMS are defined. Then several products are evaluated according to the criteria. Finally, the recommendation for the selection is decided.

Application Design

- In the **application design phase**, the design of the user interface and the application programs that use and process the database are defined and designed.

Prototyping

- The purpose of a **prototype** is to allow the users to use the prototype to identify the features of the system using the computer.
- There are horizontal and vertical prototypes. A horizontal prototype has many features (e.g. user interfaces) but they are not working. A vertical prototype has very few features but they are working. See the following picture.



Implementation

- During the **implementation phase**, the physical realization of the database and application designs are to be done. This is the programming phase of the systems development.

Data Conversion and Loading

- This phase is needed when a new database is replacing an old system. During this phase the existing data will be transferred into the new database.

Testing

- Before the new system is going to live, it should be thoroughly tested. The goal of testing is to find errors! The goal is not to prove the software is working well.

Operational Maintenance

- The **operational maintenance** is the process of **monitoring** and **maintaining** the database system.
- **Monitoring** means that the performance of the system is observed. If the performance of the system falls below an acceptable level, tuning or reorganization of the database may be required.
- **Maintaining** and upgrading the database system means that, when new requirements arise, the new development lifecycle will be done.

Database Design-Phases

The **database design phase** is divided into three steps:

1. conceptual database design
 2. logical database design
 3. physical database design
- In the **conceptual database design** phase, the model of the data to be used independent of all physical considerations is to be constructed. The model is based on the requirements specification of the system.
 - In the **logical database design** phase, the model of the data to be used is based on a specific data model, but independent of a particular database management system is constructed. This is based on the target data model for the database, e.g. relational data model.
 - In the **physical database design** phase, the description of the implementation of the database on secondary storage is created. The base relations, indexes, integrity constraints, security, etc. are defined using the SQL language.

Database Design Methodology – Overview

Conceptual Database Design

■ Step 1 Build conceptual data model

Step 1.1 Identify entity types.

Step 1.2 Identify relationship types.

Step 1.3 Identify and associate attributes with entity or relationship types.

Step 1.4 Determine attribute domains.

Step 1.5 Determine candidate, primary & alternate key attributes.

Step 1.6 Consider use of enhanced modeling concepts. (optional step)

Step 1.7 Check model for redundancy.

Step 1.8 Validate conceptual model against user transactions.

Step 1.9 Review conceptual data model with user.

Database Design Methodology – Overview

Logical Database Design for the Relational Model

- Step 2 Build logical data model
 - Step 2.1 Derive relations for local logical data model.
 - Step 2.2 Validate relations using normalization.
 - Step 2.3 Validate relations against user transactions.
 - Step 2.4 Check integrity constraints.
 - Step 2.5 Review logical data model with user.
 - Step 2.6 Merge logical data models into global model.(optional step)
 - Step 2.7 Check for future growth

Database Design Methodology – Overview

Physical Database Design for the Relational Databases

- Step 3 Translate logical data model for target DBMS
 - Step 3.1 Design base relations.
 - Step 3.2 Design representation of derived data.
 - Step 3.3 Design general constraints.
- Step 4 Design file organizations and indexes
 - Step 4.1 Analyze transactions.
 - Step 4.2 Choose file organization.
 - Step 4.3 Choose indexes.
 - Step 4.4 Estimate disk space requirements.

Database Design Methodology – Overview

Physical Database Design for the Relational Databases

- Step 5 Design user views.
- Step 6 Design security mechanisms.
- Step 7 Consider the introduction of controlled redundancy.
- Step 8 Monitor and tune the operational system.

CONCEPTUAL DATABASE DESIGN

Step 1: Build Conceptual Data Model

Objective- To build one or more conceptual data model of the data requirements of the enterprise.

This objective could be achieved by following these activities:

- Step 1.1 Identify entity types.
- Step 1.2 Identify relationship types.
- Step 1.3 Identify and associate attributes with entity or relationship types.
- Step 1.4 Determine attribute domains.
- Step 1.5 Determine candidate, primary & alternate key attributes.
- Step 1.6 Consider use of enhanced modeling concepts. (optional step)
- Step 1.7 Check model for redundancy.
- Step 1.8 Validate conceptual model against user transactions.
- Step 1.9 Review conceptual data model with user.

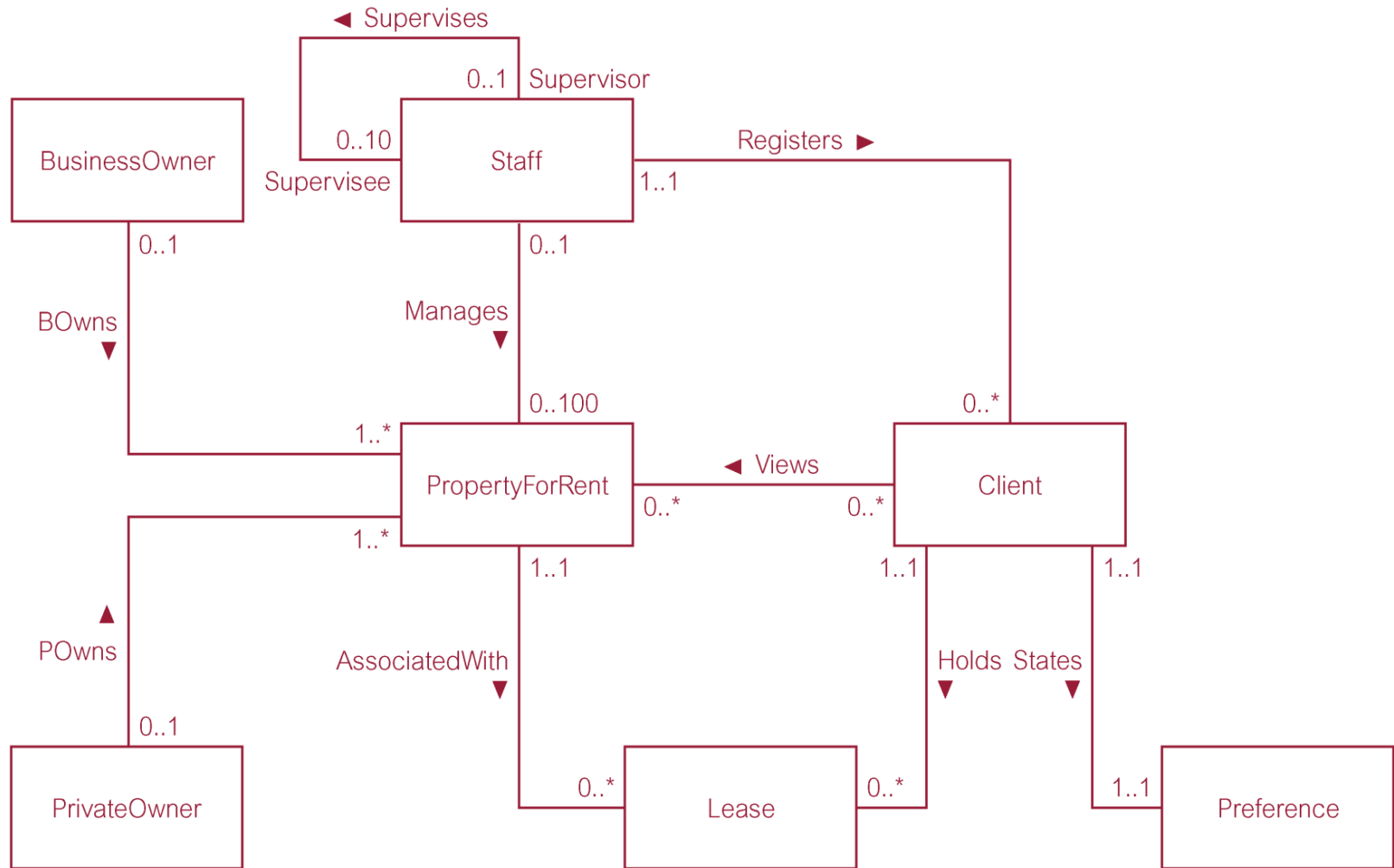
Document the entity type

Extract from the data dictionary for the StaffClient user views of *DreamHome*, showing a description of entities

<i>Entity name</i>	<i>Description</i>	<i>Aliases</i>	<i>Occurrence</i>
Staff	General term describing all staff employed by <i>DreamHome</i> .	Employee	Each member of staff works at one particular branch.
PropertyForRent	General term describing all property for rent.	Property	Each property has a single owner and is available at one specific branch, where the property is managed by one member of staff. A property is viewed by many clients and rented by a single client, at any one time.

Refer: Chapter 16, Figure 16.1,Page:472

ER diagram for the StaffClient user views of *DreamHome*



Refer: Chapter 16, Figure 16.2, Page: 474

Document the Relationship type

Extract from the data dictionary for StaffClient user views of *DreamHome* ,showing description of relationships.

<i>Entity name</i>	<i>Multiplicity</i>	<i>Relationship</i>	<i>Entity name</i>	<i>Multiplicity</i>
Staff	0..1 0..1	<i>Manages</i> <i>Supervises</i>	PropertyForRent Staff	0..100 0..10
PropertyForRent	1..1	<i>AssociatedWith</i>	Lease	0..*

Refer: Chapter 16, Figure 16.3,Page:475

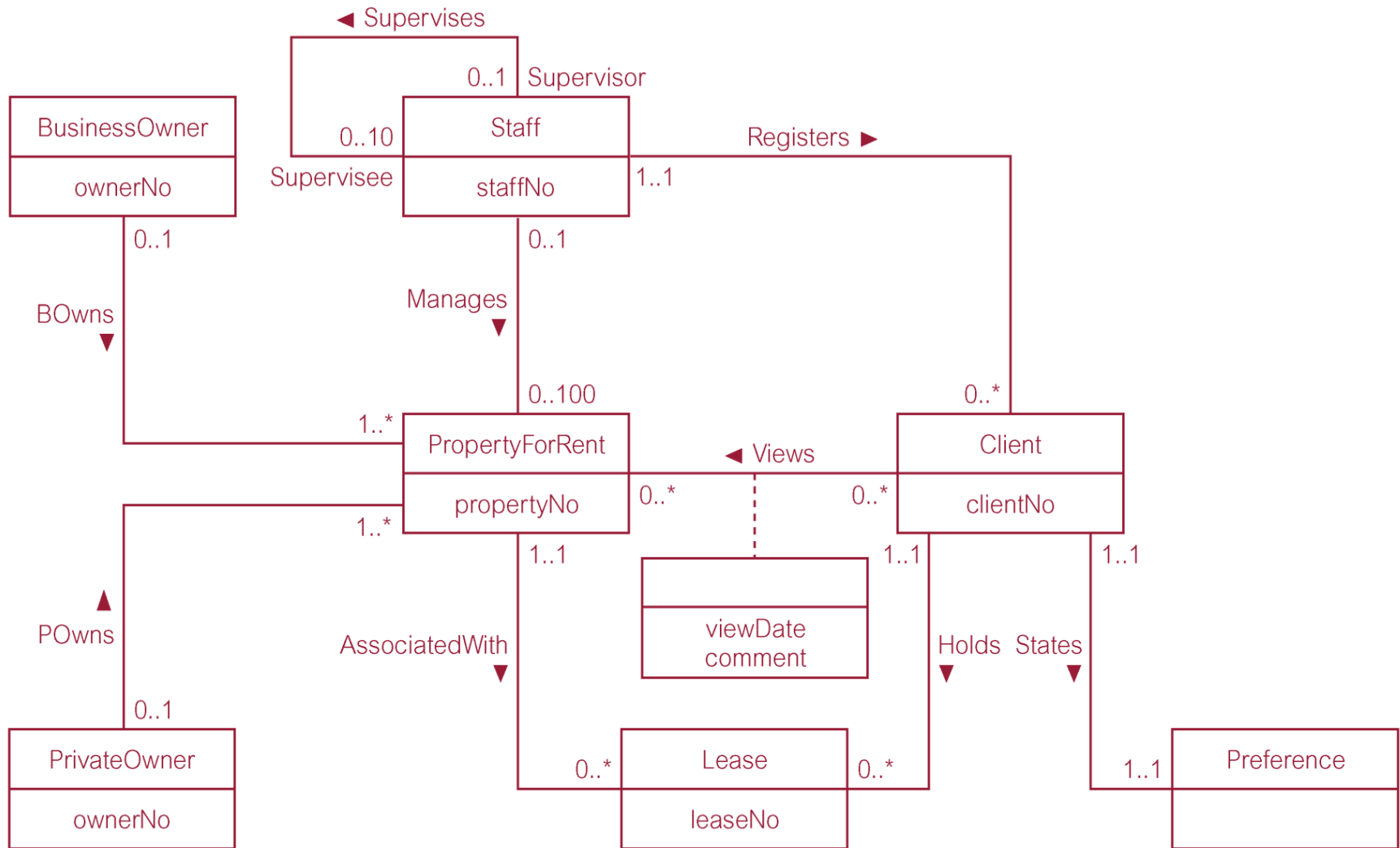
Document attributes

Extract from the data dictionary for StaffClient user views of *DreamHome* ,showing description of attributes.

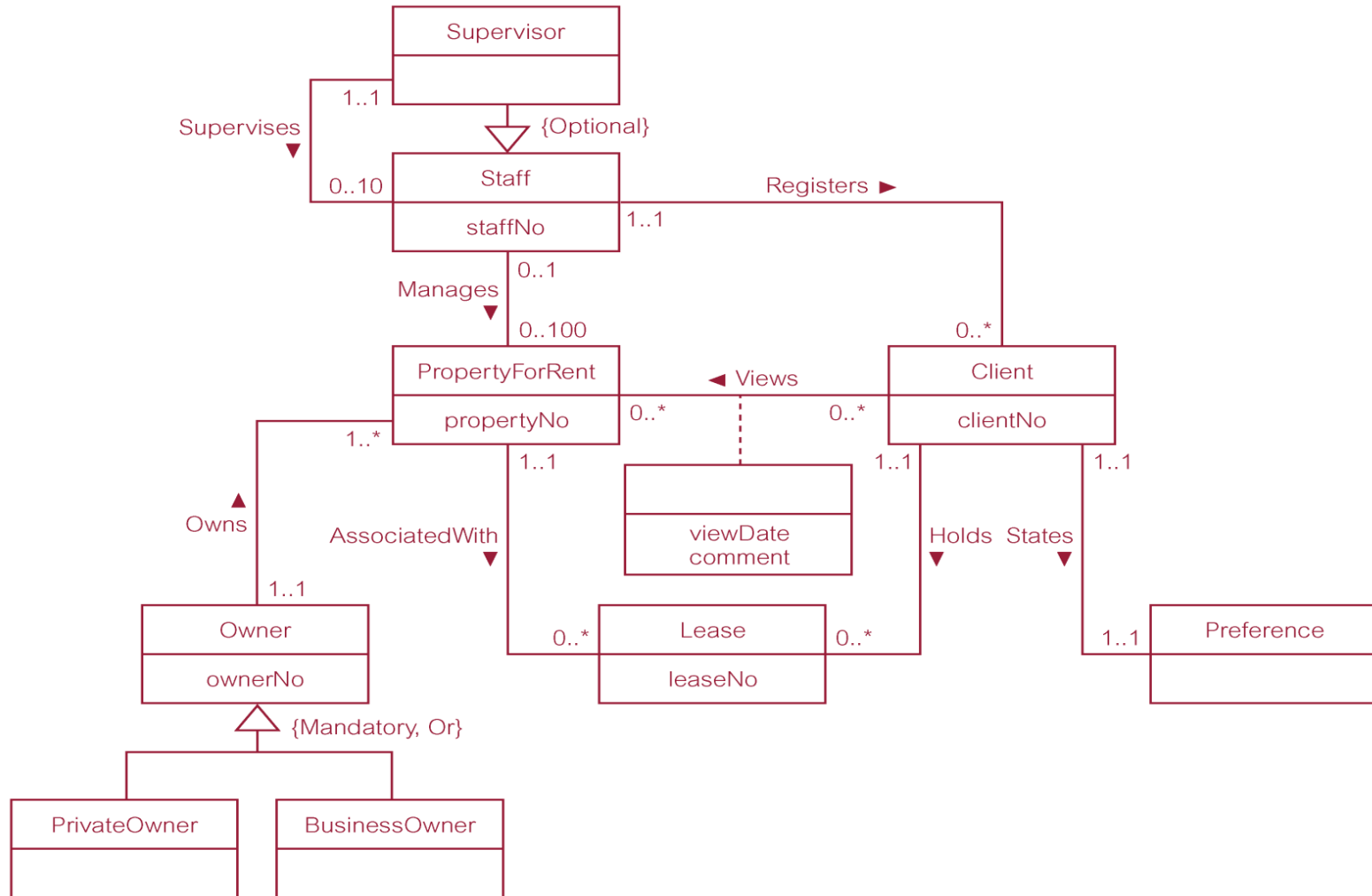
Entity name	Attributes	Description	Data Type & Length	Nulls	Multi-valued	...
Staff	staffNo	Uniquely identifies a member of staff	5 variable characters	No	No	
	name					
	fName	First name of staff	15 variable characters	No	No	
	lName	Last name of staff	15 variable characters	No	No	
	position	Job title of member of staff	10 variable characters	No	No	
	sex	Gender of member of staff	1 character (M or F)	Yes	No	
	DOB	Date of birth of member of staff	Date	Yes	No	
PropertyForRent	propertyNo	Uniquely identifies a property for rent	5 variable characters	No	No	

Refer: Chapter 16, Figure 16.4,Page:478

ER diagram for the StaffClient user views of *DreamHome* with primary keys added



ER diagram for the StaffClient user views of *DreamHome* with specialization/generalization added



Describing User Transactions

- a) List the details of staff supervised by a named supervisor at the branch.
- b) List details of all assistants alphabetically by name at the branch.
- c) List the details of property(including the rental deposit) available for rent at the branch, along with the owner's details.
- d) List the details of properties managed by a named member of staff at the branch.
- e) List the clients registering at the branch and the names of the members of staff who registered the clients.
- f) Identify properties located in Glasgow with rents no higher than £450.
- g) Identify the name and telephone number of an owner of a given property.
- h) List the details of comments made by clients viewing a given property.
- i) Display the names and phone numbers of clients who have viewed a given property but not supplied comments.
- j) Display the details of a lease between a named client and a given property.
- k) Identify the leases due to expire next month at the branch.
- l) List the details of properties that have not been rented out for more than three months.
- m) Produce a list of clients whose preferences match a particular property.

LOGICAL DATABASE DESIGN

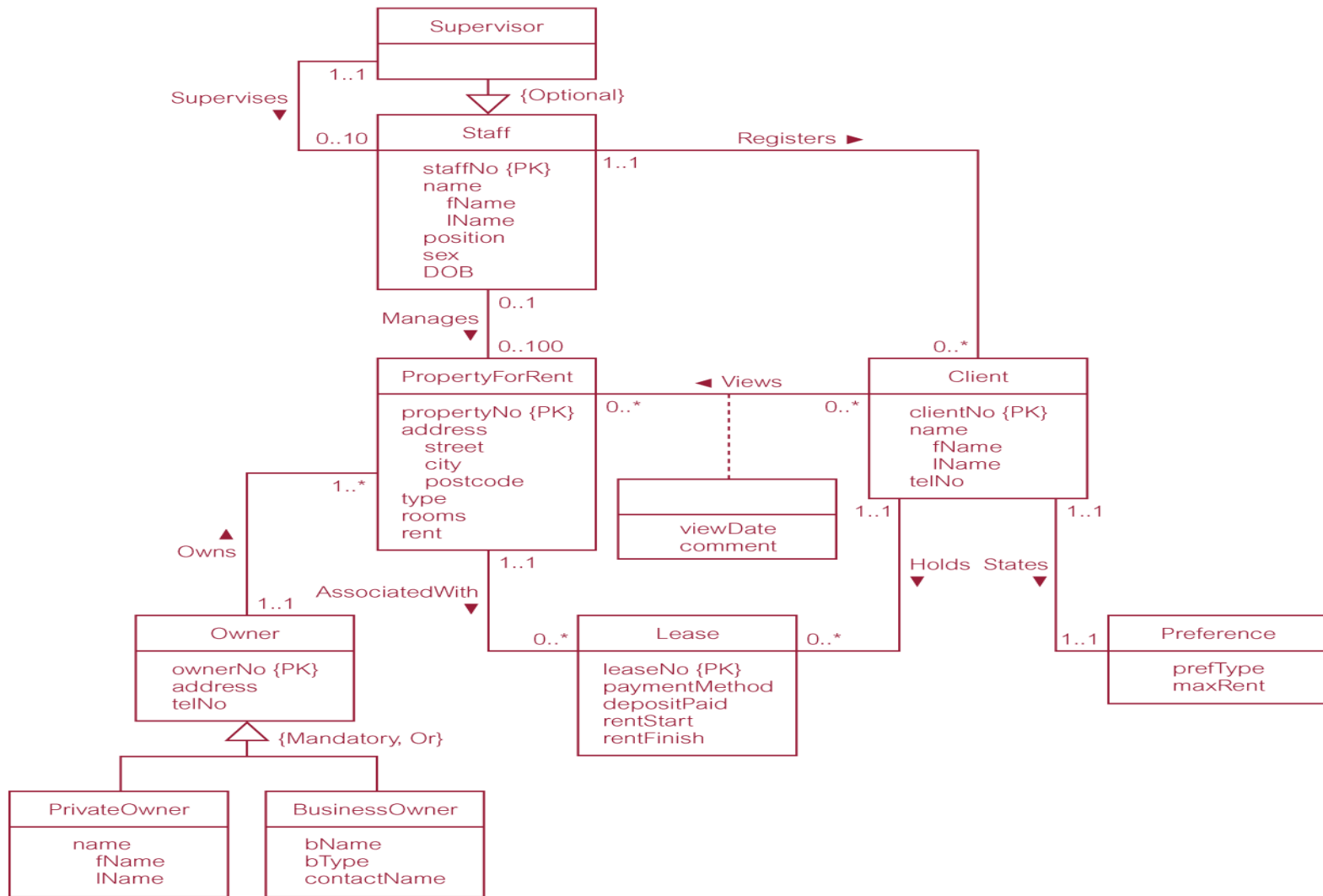
Step 2: Build Logical Data Model

Objective: To translate the conceptual data model into a logical data model, and then to validate this model to ensure it is structurally correct (using the technique of normalization) and to ensure it supports the required transactions.

This objective could be achieved by following these activities:

- ❑ Step 2.1 Derive relations for local logical data model.
- ❑ Step 2.2 Validate relations using normalization.
- ❑ Step 2.3 Validate relations against user transactions.
- ❑ Step 2.4 Check integrity constraints.
- ❑ Step 2.5 Review logical data model with user.
- ❑ Step 2.6 Merge logical data models into global model.(optional step)
- ❑ Step 2.7 Check for future growth

Conceptual data model for StaffClient user views showing all attributes



Step 2: Build Logical Data Model

Step 2.1: Derive relations for logical data model

Objective: To create relations for logical data model to represent entities, relationships, and attributes that have been identified.

- To describe how relations are derived for the following structures that may occur in a conceptual data model:
 - 1) Strong entity types;
 - 2) Weak entity types;
 - 3) One-to-many (1:*) binary relationship types;
 - 4) One-to-one (1:1) binary relationship types;
 - 5) One-to-one (1:1) recursive relationship types ;
 - 6) Superclass/subclass relationship types;
 - 7) Many-to-many (*.*) binary relationship types;
 - 8) Complex relationship types;
 - 9) Multi-valued attributes.

Step 2.1: Derive relations for logical data model

(1) Strong entity types:

For each strong entity in the data model,

- ❑ Create a relation that includes all simple attributes of that entity.
- ❑ For composite attributes, include only constituent simple attributes.

Staff (staffNo, fName, lName, position, sex, DOB)

Primary Key staffNo

Step 2.1: Derive relations for logical data model

(2) Weak entity types:

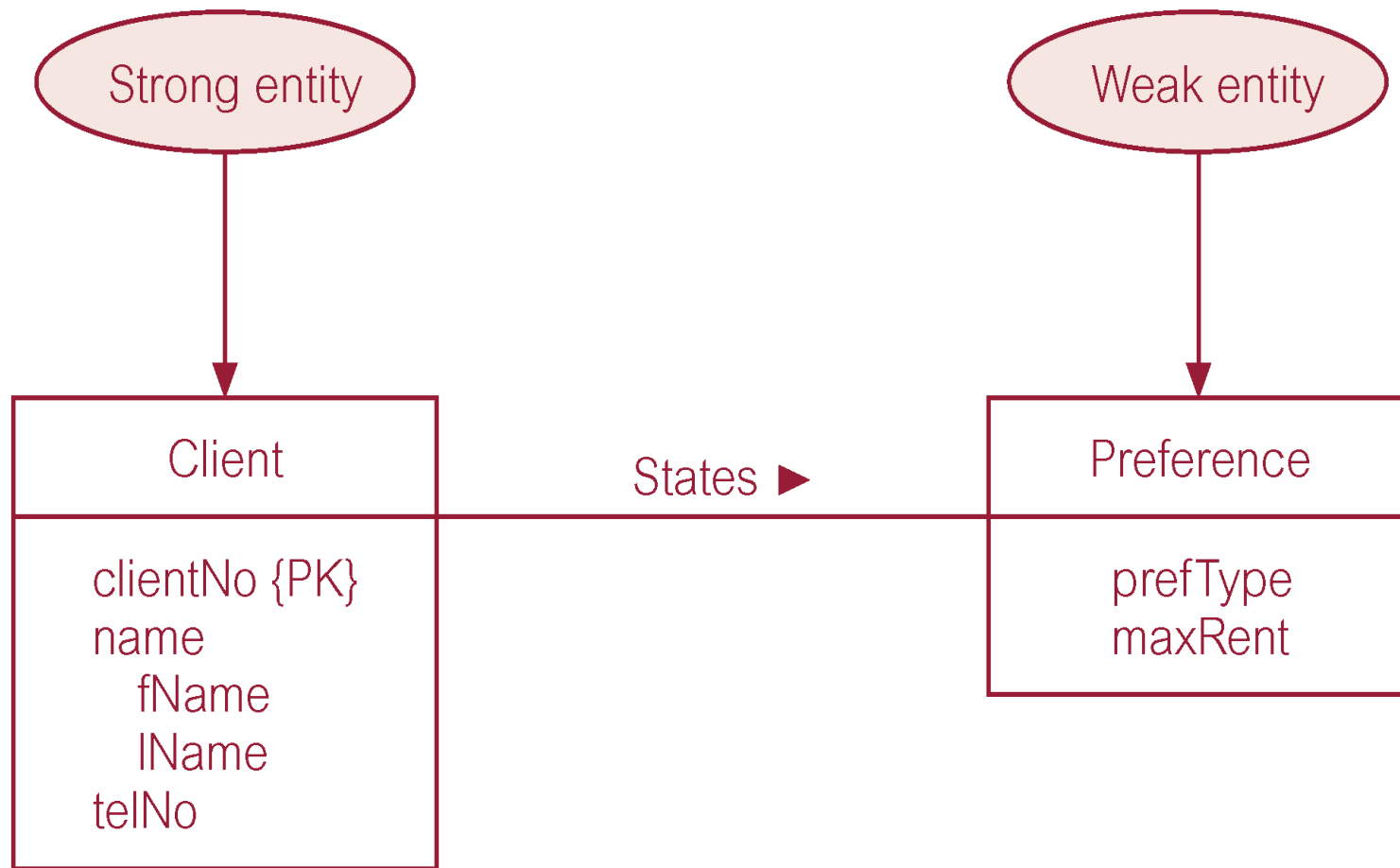
For each weak entity in the data model,

- ❑ Create a relation that includes all simple attributes of that entity.
- ❑ Primary key is partially or fully derived from each owner entity.

Preference (prefType, maxRent)

Primary Key None (at present)

Strong entity type called Client and weak entity type called Preference



Step 2.1: Derive relations for logical data model

(3) one-to many (1:*) binary relationship types:

For each 1:* binary relationship,

- Entity on 'one side' is designated as the parent entity and entity on 'many side' is designated as the child entity.
- Post a copy of the primary key attribute(s) of parent entity into relation representing child entity, to act as a foreign key.

Post **staffNo** into **Client** to model 1:* *Registers* relationship

Staff (staffNo, fName, lName, position, sex, DOB)

Primary Key staffNo

Client (clientNo, fName, lName, telNo, staffNo)

Primary Key clientNo

Alternate Key telNo

Foreign Key staffNo references Staff(staffNo)

Step 2.1: Derive relations for logical data model

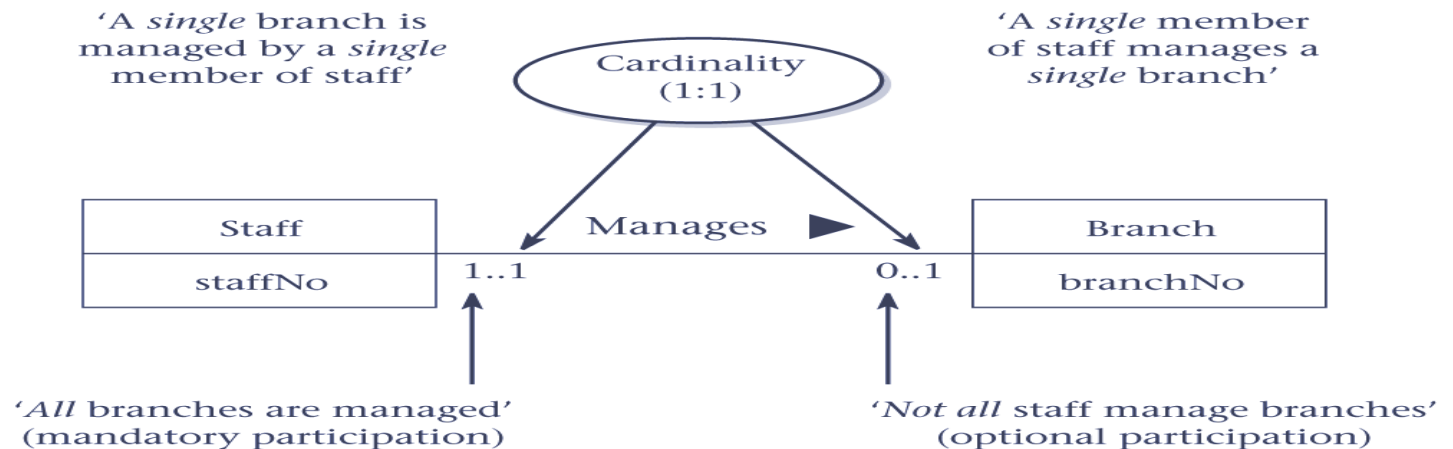
(4) One-to-one (1:1) binary relationship types:

Creating relations to represent 1:1 relationship is slightly more complex,

- ❑ Cardinality cannot be used to identify parent and child entities in a relationship.
- ❑ Instead, participation are used to decide whether to combine entities into one relation or to create two relations and post a copy of primary key from one relation to the other. Consider the following:
 - (a) *mandatory* participation on *both* sides of 1:1 relationship;
 - (b) *mandatory* participation on *one* side of 1:1 relationship;
 - (c) *optional* participation on *both* sides of 1:1 relationship.

Cardinality and Participation (re-visit)

- **Cardinality** describes the number of possible relationships for each participating entity.
- **Participation** determines whether all or only some entity occurrences participate in a relationship.
- The cardinality of a binary relationship is what we have been referring to as one-to-one, one-to-many, and many-to-many.
- A participation constraint represents whether all entity occurrences are involved in a particular relationship (*mandatory participation*) or only some (*optional participation*).
- The cardinality and participation constraints for the Staff Manages Branch relationship are shown below.



Step 2.1: Derive relations for logical data model

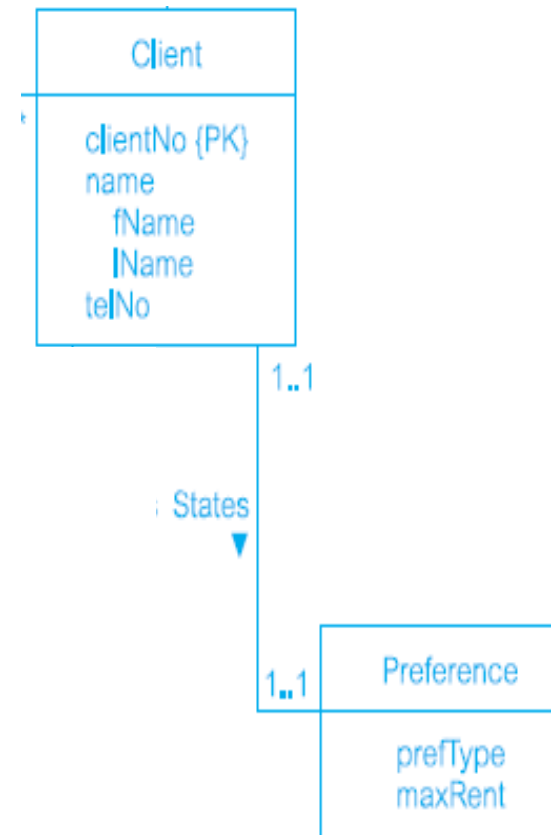
(a) *Mandatory* participation on *both* sides of 1:1 relationship

- ❑ Combine entities involved into one relation
- ❑ choose one of the primary keys of original entities to be primary key of new relation
- ❑ other (if one exists) is used as an alternate key.

Client (clientNo, fName, lName, telNo, prefType, maxRent, staffNo)

Primary Key: clientNo

Foreign Key: staffNo references Staff(staffNo)



Step 2.1: Derive relations for logical data model

(b) *Mandatory* participation on *one* side of a 1:1 relationship:

In this case, we are able to,

- ❑ Identify parent and child entities using participation constraints.
- ❑ Entity with optional participation is designated parent entity, and entity with mandatory participation is designated child entity.
- ❑ Copy of primary key of parent entity is placed in relation representing child entity.
- ❑ If relationship has one or more attributes, these attributes should follow the posting of the primary key to the child relation.

For 1:1 relationship with mandatory participation on **Client** side, post **clientNo** into **Preference** to model **States** relationship

Client (clientNo, fName, lName, telNo, staffNo)

Primary Key clientNo

Foreign Key staffNo **references** Staff(staffNo)

Preference (clientNo, prefType, maxRent)

Primary Key clientNo

Foreign Key clientNo **references** Client(clientNo)

Step 2.1: Derive relations for logical data model

(c) *Optional* participation on *both* sides of a 1:1 relationship:

In this case,

- ❑ Designation of the parent and child entities is arbitrary unless can find out more about the relationship.
- Consider 1:1 Staff *Uses* Car relationship with optional participation on both sides. Assume majority of cars, but not all, are used by staff and only minority of staff use cars.
- Car entity, although optional, is closer to being mandatory than Staff entity. Therefore designate Staff as parent entity and Car as child entity.

Step 2.1: Derive relations for logical data model

(5) 1:1 recursive relationships - follow rules for participation for a 1:1 relationship:

- ❑ mandatory participation on both sides: single relation with two copies of the primary key.
- ❑ One copy $\Rightarrow$ primary key; other copy $\Rightarrow$ foreign key, should be renamed
- ❑ mandatory participation on only one side:
 - option to create a single relation with two copies of the primary key, or
 - create a new relation to represent the relationship. The new relation would only have two attributes, both copies of the primary key.
- ❑ optional participation on both sides, create a new relation as described above.

Step 2.1: Derive relations for logical data model

(6) Superclass/subclass relationship types:

- ❑ Identify superclass as parent entity and subclass entity as child entity.
- ❑ There are various options on how to represent such a relationship as one or more relations.
- ❑ Most appropriate option dependent on number of factors such as:
 - disjointness and participation constraints on the superclass/subclass relationship,
 - whether subclasses are involved in distinct relationships,
 - number of participants in superclass/subclass relationship.

Step 2.1: Derive relations for logical data model

(7) ***:* binary relationship types**

- ❑ Create relation to represent relationship and include any attributes that are part of relationship.
- ❑ Post a copy of the primary key attribute(s) of the entities that participate in relationship into new relation, to act as foreign keys.
- ❑ These foreign keys will also form primary key of new relation, possibly in combination with some of the attributes of the relationship.

Step 2.1: Derive relations for logical data model

(7) **: binary relationship types - Example

Client (clientNo, fName, lName, telNo, prefType, maxRent, staffNo)

Primary Key clientNo Foreign Key staffNo references Staff(staffNo)

PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent)

Primary Key propertyNo

Viewing (clientNo, propertyNo, dateView, comment)

Primary Key clientNo, propertyNo

Foreign Key clientNo references Client(clientNo)

Foreign Key propertyNo references PropertyForRent(propertyNo)

Step 2.1: Derive relations for logical data model

(8) Complex relationship types

- ❑ Create relation to represent relationship and include any attributes that are part of the relationship.
- ❑ Post copy of primary key attribute(s) of entities that participate in the complex relationship into new relation, to act as foreign keys.
- ❑ Any foreign keys that represent a 'many' relationship (for example, 1..*, 0..*) generally will also form the primary key of new relation, possibly in combination with some of the attributes of the relationship.

Step 2.1: Derive relations for logical data model

(8) Complex relationship types - Example

Staff (staffNo, fName, lName, position, sex, DOB, supervisorStaffNo)

Primary Key staffNo

Foreign Key supervisorStaffNo **references** Staff(staffNo)

Branch (branchNo, street, city, postcode)

Primary Key branchNo

Client (clientNo, fName, lName, telNo, prefType, maxRent, staffNo)

Primary Key clientNo

Foreign Key staffNo **references** Staff(staffNo)

Registration (clientNo, branchNo, staffNo, dateJoined)

Primary Key clientNo

Foreign Key branchNo **references** Branch(branchNo)

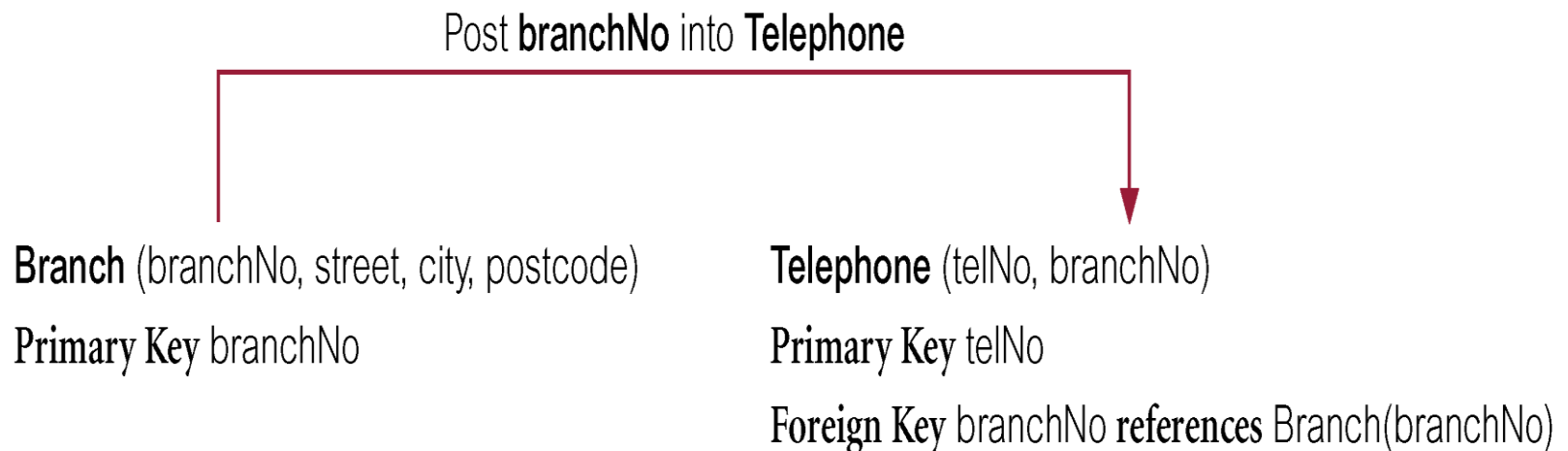
Foreign Key clientNo **references** Client(clientNo)

Foreign Key staffNo **references** Staff(staffNo)

Step 2.1: Derive relations for logical data model

(9) Multi-valued attributes

- ❑ Create new relation to represent multi-valued attribute and include primary key of entity in new relation, to act as a foreign key.
- ❑ Unless the multi-valued attribute is itself an alternate key of the entity, primary key of new relation is combination of the multi-valued attribute and the primary key of the entity.



Relations for the Staff view of *DreamHome*

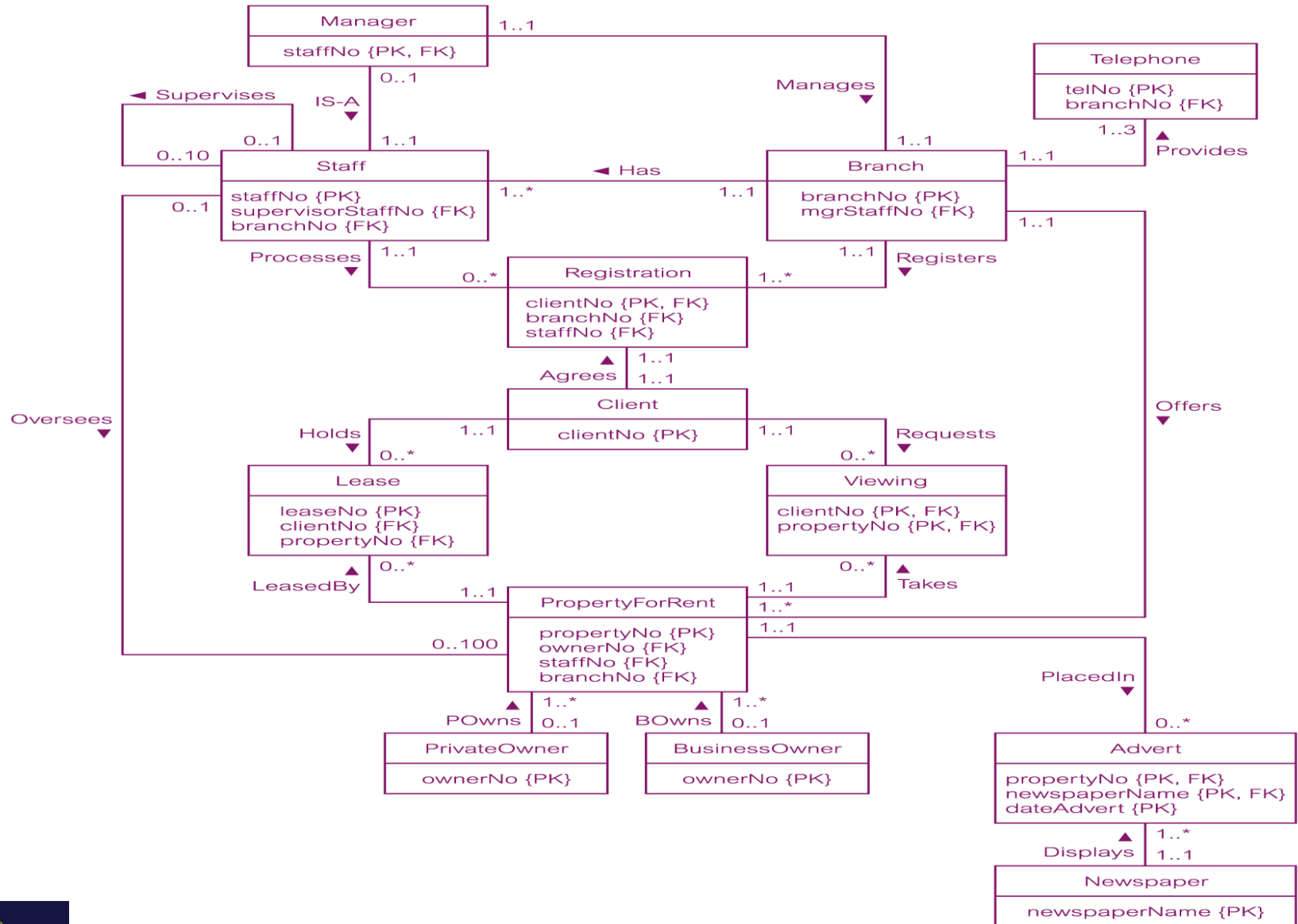
Staff (staffNo, fName, lName, position, sex, DOB, supervisorStaffNo) Primary Key staffNo Foreign Key supervisorStaffNo references Staff(staffNo)	PrivateOwner (ownerNo, fName, lName, address, telNo) Primary Key ownerNo
BusinessOwner (ownerNo, bName, bType, contactName, address, telNo) Primary Key ownerNo Alternate Key bName Alternate Key telNo	Client (clientNo, fName, lName, telNo, prefType, maxRent, staffNo) Primary Key clientNo Foreign Key staffNo references Staff(staffNo)
PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo, staffNo) Primary Key propertyNo Foreign Key ownerNo references PrivateOwner(ownerNo) and BusinessOwner(ownerNo) Foreign Key staffNo references Staff(staffNo)	Viewing (clientNo, propertyNo, dateView, comment) Primary Key clientNo, propertyNo Foreign Key clientNo references Client(clientNo) Foreign Key propertyNo references PropertyForRent(propertyNo)
Lease (leaseNo, paymentMethod, depositPaid, rentStart, rentFinish, clientNo, propertyNo) Primary Key leaseNo Alternate Key propertyNo, rentStart Alternate Key clientNo, rentStart Foreign Key clientNo references Client(clientNo) Foreign Key propertyNo references PropertyForRent(propertyNo) Derived deposit (PropertyForRent.rent*2) Derived duration (rentFinish – rentStart)	

Relations that represent the global logical data model for *DreamHome*

Branch (branchNo, street, city, postcode, mgrStaffNo) Primary Key branchNo Alternate Key postcode Foreign Key mgrStaffNo references Manager(staffNo)	Telephone (telNo, branchNo) Primary Key telNo Foreign Key branchNo references Branch(branchNo)
Staff (staffNo, fName, lName, position, sex, DOB, salary, supervisorStaffNo, branchNo) Primary Key staffNo Foreign Key supervisorStaffNo references Staff(staffNo) Foreign Key branchNo references Branch(branchNo)	Manager (staffNo, mgrStartDate, bonus) Primary Key staffNo Foreign Key staffNo references Staff(staffNo)
PrivateOwner (ownerNo, fName, lName, address, telNo) Primary Key ownerNo	BusinessOwner (ownerNo, bName, bType, contactName, address, telNo) Primary Key ownerNo Alternate Key bName Alternate Key telNo
PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo, staffNo, branchNo) Primary Key propertyNo Foreign Key ownerNo references PrivateOwner(ownerNo) and BusinessOwner(ownerNo) Foreign Key staffNo references Staff(staffNo) Foreign Key branchNo references Branch(branchNo)	Viewing (clientNo, propertyNo, dateView, comment) Primary Key clientNo, propertyNo Foreign Key clientNo references Client(clientNo) Foreign Key propertyNo references PropertyForRent(propertyNo)
Client (clientNo, fName, lName, telNo, prefType, maxRent) Primary Key clientNo	Registration (clientNo, branchNo, staffNo, dateJoined) Primary Key clientNo Foreign Key clientNo references Client(clientNo) Foreign Key branchNo references Branch(branchNo) Foreign Key staffNo references Staff(staffNo)
Lease (leaseNo, paymentMethod, depositPaid, rentStart, rentFinish, clientNo, propertyNo) Primary Key leaseNo Alternate Key propertyNo, rentStart Alternate Key clientNo, rentStart Foreign Key clientNo references Client(clientNo) Foreign Key propertyNo references PropertyForRent(propertyNo) Derived deposit (PropertyForRent.rent*2) Derived duration (rentFinish – rentStart)	Newspaper (newspaperName, address, telNo, contactName) Primary Key newspaperName Alternate Key telNo
Advert (propertyNo, newspaperName, dateAdvert, cost) Primary Key propertyNo, newspaperName, dateAdvert Foreign Key propertyNo references PropertyForRent(propertyNo) Foreign Key newspaperName references Newspaper(newspaperName)	



Global relation diagram for *DreamHome*



The End

Thank You